Tesla_Regression_Analysis

October 5, 2026

1 Predicting Tesla's Next-Day Closing Price

DATA 555 | Project 1: Regression Analysis

Authors: Silvana Stipetich (CCID: 1951614); Felipe Costa (CCID: 1935821)

Goal: Can we predict tomorrow's closing price of Tesla stock using only what we know at today's close? We try four regression models (simple linear, multiple linear, degree-2 polynomial and Ridge) and compare them with the simplest possible guess: tomorrow's price will be the same as today's.

Data: Tesla Stock Data from 2010 to 2020, from Kaggle (user *timoboz*). The file `TSLA.csv` has one row per trading day from 29 June 2010 to 3 February 2020.

How to run: Keep `TSLA.csv` in the same folder as this notebook and run the cells from top to bottom. You will need pandas, numpy, matplotlib, seaborn, statsmodels and scikit-learn.

1.1 1. Load, check and prepare the data

First we look at the data and check its quality: data types, missing values, duplicates and whether the prices make sense (for example, the daily high should never be below the low). Then we sort by date and build the variables for the model. The target is the closing price of the next row in the file, which is the next trading day. The last day has no "next day", so we drop it.

[1]:
```
# Import libs
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm
import seaborn as sns

from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV, TimeSeriesSplit
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

[2]:
```
# Global configs
pd.set_option('display.width', 130)
plt.rcParams.update({'figure.figsize': (9, 3.5), 'axes.spines.top': False,␣
 ↪'axes.spines.right': False})
```

[3]:
```
# Read data
df = pd.read_csv('TSLA.csv')

# Print some informations about the data
print(df.head().to_string(index=False))
print('\nRows and columns:', df.shape)
print('Data types:', df.dtypes.astype(str).to_dict())
print('Missing values:', df.isna().sum().to_dict())
print('Duplicate rows:', df.duplicated().sum())

# Convert object to data format.
df['Date'] = pd.to_datetime(df['Date'], errors='raise')
# Sort by date
df = df.sort_values('Date').reset_index(drop=True)
# How many duplicate date there are
print('Duplicate dates:', df.Date.duplicated().sum())
```

```
      Date       Open  High       Low     Close  Adj Close   Volume
2010-06-29 19.000000 25.00 17.540001 23.889999  23.889999 18766300
2010-06-30 25.790001 30.42 23.299999 23.830000  23.830000 17187100
2010-07-01 25.000000 25.92 20.270000 21.959999  21.959999  8218800
2010-07-02 23.000000 23.10 18.709999 19.200001  19.200001  5139800
2010-07-06 20.000000 20.00 15.830000 16.110001  16.110001  6866900

Rows and columns: (2416, 7)
Data types: {'Date': 'object', 'Open': 'float64', 'High': 'float64', 'Low':
'float64', 'Close': 'float64', 'Adj Close': 'float64', 'Volume': 'int64'}
Missing values: {'Date': 0, 'Open': 0, 'High': 0, 'Low': 0, 'Close': 0, 'Adj
Close': 0, 'Volume': 0}
Duplicate rows: 0
Duplicate dates: 0
```

[4]:
```
# Is there any invalid value (e.g. low > high)
valid_prices = (df.High >= df[['Open','Low','Close']].max(axis=1)) & (df.Low <=␣
 ↪df[['Open','High','Close']].min(axis=1))
assert not df.isna().any().any() and not df.Date.duplicated().any()
assert valid_prices.all() and (df.Volume >= 0).all()
assert (df[['Open','High','Low','Close','Adj Close']] > 0).all().all()
print('Price and volume consistency checks: passed')
print('Close equals adjusted close:', np.allclose(df.Close, df['Adj Close']))
print('Dates:', df.Date.min().date(), 'to', df.Date.max().date())

# Data value conversion
df['Volume_M'] = df.Volume / 1_000_000
df['Range'] = df.High - df.Low
df['Next_Close'] = df.Close.shift(-1)
df['Target_Date'] = df.Date.shift(-1)
```

```
# Drop nan data in the column Next_Close
data = df.dropna(subset=['Next_Close']).copy()
features = ['Close','Volume_M','Range']

print('\nPrepared observations:', len(data))
print(data[features+['Next_Close']].describe().round(2).to_string())
```

```
Price and volume consistency checks: passed
Close equals adjusted close: True
Dates: 2010-06-29 to 2020-02-03

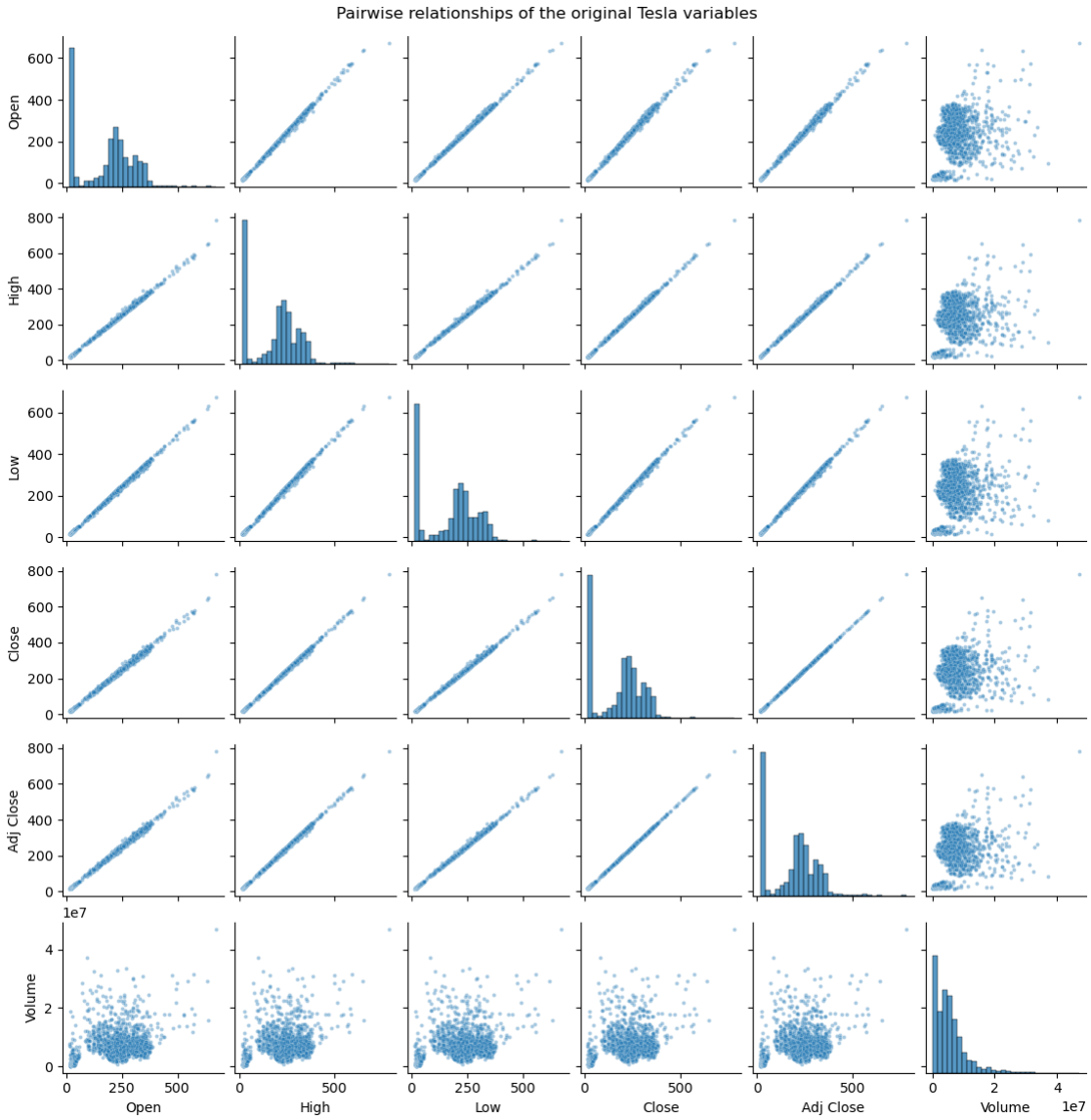
Prepared observations: 2415
         Close  Volume_M    Range  Next_Close
count  2415.00   2415.00  2415.00     2415.00
mean    186.16      5.56     6.62      186.47
std     118.55      4.92     5.38      119.11
min      15.80      0.12     0.25       15.80
25%      34.40      1.90     1.88       34.40
50%     212.96      4.58     5.85      212.96
75%     266.72      7.36     9.25      266.78
max     650.57     37.16    48.31      780.00
```

The data is already clean: no missing values, no duplicates and no impossible prices, so we did not need to fill in or remove anything. We also noticed that `Adj Close` is exactly the same as `Close`, so we only keep `Close`. After creating the next-day target we have 2,415 rows. We added two predictors: volume in millions of shares (`Volume_M`) and the daily range, which is the high minus the low (`Range`).

1.2 2. Exploratory analysis of the original variables

We start with a pairplot because it puts every pair of variables side by side in one figure, together with the histogram of each variable. It is a quick way to see which columns move together before deciding which ones to use as predictors. Here we use the full dataset, since this is only a first description.

[5]:
```
# Pairwise relationships and distributions of the original numeric columns
sns.pairplot(df[['Open','High','Low','Close','Adj Close','Volume']], height=1.9,
             plot_kws={'s':8,'alpha':0.4}, diag_kws={'bins':30})
plt.suptitle('Pairwise relationships of the original Tesla variables', y=1.01)
plt.show()
```



The five price columns (Open, High, Low, Close and Adj Close) lie almost on a perfect straight line, so they basically tell the same story. Putting them all in the same model would cause strong multicollinearity, so we only keep `Close` data. The histograms have two peaks: about 29% of the days (from 2010 to April 2013) traded below USD 50, while most of the later days traded between USD 150 and 400. Volume data is skewed to the right and its relationship with price is much weaker and more scattered.

1.3 3. Split the data by date

We train the models on the first 80% of the days and test them on the last 20%. Since this is a time series, we keep the dates in order instead of shuffling them; otherwise the model would "see the future" during training. We also leave out one row at the boundary so that no training target falls inside the test period. All models use exactly the same training and test days.

[6]:
```
# Interval selected to train data
cut = int(len(data)*0.8)
train, test = data.iloc[:cut-1].copy(), data.iloc[cut:].copy()
assert train.Target_Date.max() < test.Date.min()

print('Training:', len(train), '| Omitted boundary row: 1 | Testing:',␣
 ↪len(test))
print('Training targets:', train.Target_Date.min().date(), 'to', train.
 ↪Target_Date.max().date())
print('Test targets:', test.Target_Date.min().date(), 'to', test.Target_Date.
 ↪max().date())
```

```
Training: 1931 | Omitted boundary row: 1 | Testing: 483
Training targets: 2010-06-30 to 2018-03-01
Test targets: 2018-03-05 to 2020-02-03
```

[7]:
```
# Plots
fig, ax = plt.subplots()
ax.plot(df.Date, df.Close, linewidth=1)
ax.axvline(test.Date.min(), color='darkorange', linestyle='--', label='Test␣
 ↪begins')
ax.set(xlabel='Date', ylabel='Closing price (USD)', title='Historical closing␣
 ↪price')
ax.legend(); fig.tight_layout()
```



The plot shows a big jump in the price at the very end of the test period (early 2020), with prices much higher than anything in the training data. We will come back to this in Section 8. Note that we fit each model only once and then, for every test day, we use that day's values to predict the next day's close.

1.4 4. Relationships between predictors and the target (training data)

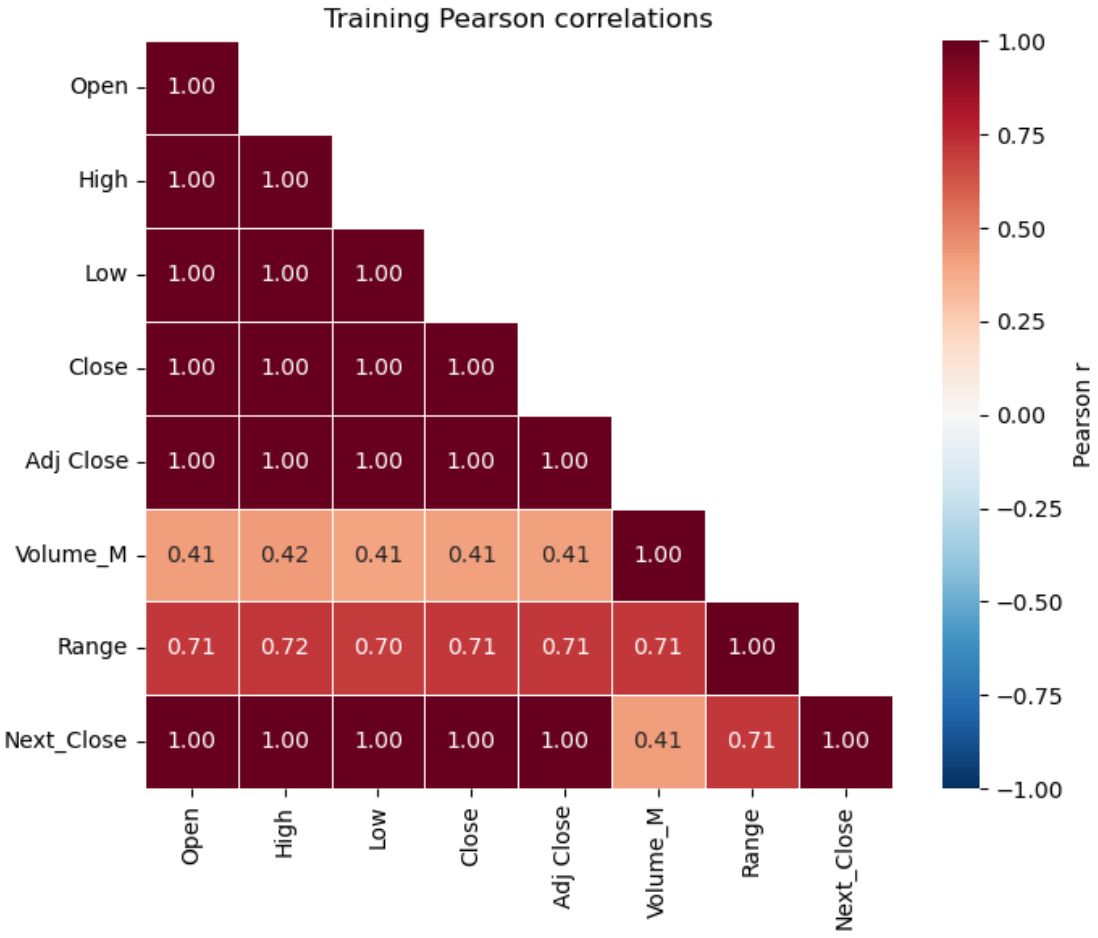
Now we look more closely at the three predictors we chose. The scatter plots show the shape of each relationship with the next-day close price (straight, curved or just scattered). The heatmap shows all the Pearson correlations at once with colours, which is much easier to read than a table of numbers. We only use training data here so that the test period does not influence our choices.

[8]:
```
# Scatter plots: each selected predictor against the target
fig, axes = plt.subplots(1,3,figsize=(11,3.5))
for ax, col in zip(axes,features):
    ax.scatter(train[col],train.Next_Close,s=8,alpha=.35)
    ax.set(xlabel=col, ylabel='Next close (USD)')
fig.suptitle('Training predictor relationships');fig.tight_layout()
plt.show()
```



Today's close price and tomorrow's close price form an almost perfect straight line. Volume and range also go up with price, but the points are much more spread out, especially at higher prices, so they probably add little extra information. We don't see a clear tred curve, so the polynomial model is more of a test than something the data asks for.

[9]:
```
# Pearson correlation heatmap (training rows only)
corr_cols = ['Open','High','Low','Close','Adj␣
 ↪Close','Volume_M','Range','Next_Close']
corr = train[corr_cols].corr()
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)   # show the lower triangle␣
 ↪only
fig, ax = plt.subplots(figsize=(7.5,6))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='RdBu_r', vmin=-1,␣
 ↪vmax=1,
            square=True, linewidths=.5, cbar_kws={'label':'Pearson r'}, ax=ax)
ax.set_title('Training Pearson correlations')
fig.tight_layout(); plt.show()
```



The dark red block confirms what the pairplot showed: all price columns, including tomorrow's close, are almost perfectly correlated (r 1.00). This means that today's close alone already explains almost all of tomorrow's price level. Range (r 0.71) and volume (r 0.41) are only moderately related to the target, and they are also related to each other (r 0.71). We keep in mind that correlation is not causation, and that a strong link between consecutive prices does not mean we can predict how much the price will change from one day to the next.

1.5 5. Fit the four models

We tried four different strategies:

- **Simple linear:** today's close only.
- **Multiple linear:** today's close, volume and range.
- **Polynomial (degree 2):** the same three predictors plus their squares (no interaction terms).
- **Ridge:** the same three predictors as the multiple model, but with a penalty that shrinks the coefficients.

The first three models are fitted with OLS in `statsmodels`, as in the course notebooks. For Ridge we first standardize the predictors and then choose the penalty (alpha) with time-ordered

cross-validation on the training data only. The test data is never used to choose alpha.

[10]:
```
def ols_inputs(frame, columns, squared=False):
    '''This function prepares the regressor matrix X
    for an ordinary least squares (OLS) regression,
    with an optional nonlinear (squared) transformation
    of selected columns.
    '''
    X = frame[columns].copy()
    if squared:
        for column in columns:
            X[column+'_squared'] = X[column]**2
    return sm.add_constant(X, has_constant='add')
```

[11]:
```
# Type of regression
specs = {'Simple linear': (['Close'],False),
         'Multiple linear': (features,False),
         'Polynomial degree 2': (features,True)}

models, predictions, fitted = {}, {}, {}

for name,(columns,squared) in specs.items():
    model = sm.OLS(train.Next_Close,ols_inputs(train,columns,squared)).fit()
    models[name] = model
    predictions[name] = model.predict(ols_inputs(test,columns,squared)).
 ↪to_numpy()
    fitted[name] = model.fittedvalues.to_numpy()

# Grid search
ridge_search = GridSearchCV(
    Pipeline([('scale',StandardScaler()),('model',Ridge())]),
    {'model__alpha': np.logspace(-4,4,25)},
    cv=TimeSeriesSplit(n_splits=5,gap=1),
    scoring='neg_mean_squared_error', n_jobs=1)

ridge_search.fit(train[features],train.Next_Close)

ridge = ridge_search.best_estimator_

predictions['Ridge'] = ridge.predict(test[features])
fitted['Ridge'] = ridge.predict(train[features])

print('Selected Ridge alpha:', ridge_search.best_params_['model__alpha'])
print('Ridge validation RMSE:', round(np.sqrt(-ridge_search.best_score_),4))
print('\nOLS coefficients:')
print(pd.DataFrame({name:model.params for name,model in models.items()}).
 ↪round(7).fillna('').to_string())
```

```
# Convert Ridge coefficients back to original predictor units for comparison.
scale = ridge.named_steps['scale']
ridge_coef = ridge.named_steps['model'].coef_ / scale.scale_
ridge_intercept = ridge.named_steps['model'].intercept_ - np.
  ↪dot(ridge_coef,scale.mean_)

print('\nRidge coefficients in original predictor units:')
print(pd.Series([ridge_intercept]+list(ridge_coef),index=['const']+features).
  ↪round(7).to_string())
```

```
Selected Ridge alpha: 0.0001
Ridge validation RMSE: 5.5992

OLS coefficients:
                 Simple linear Multiple linear  Polynomial degree 2
Close                 0.999274        1.000876             1.012680
Close_squared                                              -0.000030
Range                                -0.115818            -0.267332
Range_squared                                               0.006242
Volume_M                              0.10539             0.113873
Volume_M_squared                                           -0.000455
const                 0.272988         0.16635            -0.081575

Ridge coefficients in original predictor units:
const       0.166357
Close       1.000876
Volume_M    0.105389
Range      -0.115816
```

The coefficients show associations, not causes. In the multiple model, the coefficient for `Close` price is practically 1, which means the model is basically saying "tomorrow will be like today". Volume is measured in millions of shares and range in USD. Ridge chose alpha = 0.0001, the smallest value we tried, so it barely shrinks the coefficients. That is why its coefficients and results are almost the same as the multiple linear model.

1.5.1 OLS summary of the multiple linear model

As we did in class, we print the full `statsmodels` summary to look at R^2, adjusted R^2, the F-statistic, the coefficients and their p-values.

[12]:
```
print(models['Multiple linear'].summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:             Next_Close   R-squared:                       0.998
Model:                            OLS   Adj. R-squared:                  0.998
Method:                 Least Squares   F-statistic:                 3.257e+05
Date:                Mon, 05 Oct 2026   Prob (F-statistic):               0.00
```

```
Time:                        13:40:17   Log-Likelihood:                -5808.3
No. Observations:                1931   AIC:                         1.162e+04
Df Residuals:                    1927   BIC:                         1.165e+04
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.1663      0.203      0.818      0.413      -0.232       0.565
Close          1.0009      0.001    688.190      0.000       0.998       1.004
Volume_M       0.1054      0.038      2.752      0.006       0.030       0.180
Range         -0.1158      0.047     -2.453      0.014      -0.208      -0.023
==============================================================================
Omnibus:                      249.180   Durbin-Watson:                   1.937
Prob(Omnibus):                  0.000   Jarque-Bera (JB):             2583.143
Skew:                          -0.163   Prob(JB):                         0.00
Kurtosis:                       8.657   Cond. No.                         351.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

On the training data, the model explains almost all of the variation in the next-day price ($R^2 = 0.998$). The `Close` price coefficient is about 1.00 ($p < 0.001$). Volume ($p = 0.006$) and range ($p = 0.014$) are statistically significant at the 5% level, but their effects are tiny: about 0.11 USD per million shares and -0.12 USD per dollar of range, while the stock trades at hundreds of dollars. We should not rely too much on these p-values.

The Jarque-Bera test shows that the residuals are not normal (they have heavy tails), and the residual plots in Section 7 show that their spread is not constant. The Durbin-Watson value (1.94) suggests there is little autocorrelation. Being significant in training does not guarantee better predictions on new data, which is what we check next.

1.6 6. Evaluate the predictions

We compare the models on the test set using R^2, RMSE and MAE. RMSE and MAE are measured in USD because they are averages of the prediction errors, and each error is the actual price minus the predicted price. For example, an MAE of USD 7.64 means the predictions miss the real closing price by about 7.64 dollars per day on average.

We also add a **no-change baseline**, which simply predicts that tomorrow's close price will equal today's close price. A model is only useful if it beats this simple guess. The training RMSE helps us spot overfitting, but the test RMSE is what we use to choose the best model.

[13]:
```python
def evaluate(actual,predicted):
    '''Computes three common regression evaluation
    metrics from the true values (actual) and the model's
    predictions (predicted).
```

```
    '''
    actual,predicted = np.asarray(actual),np.asarray(predicted)
    error = actual-predicted
    return [1-np.sum(error**2)/np.sum((actual-actual.mean())**2),
            np.sqrt(np.mean(error**2)),np.mean(np.abs(error))]
```

[14]:
```
predictions['No-change baseline'] = test.Close.to_numpy()

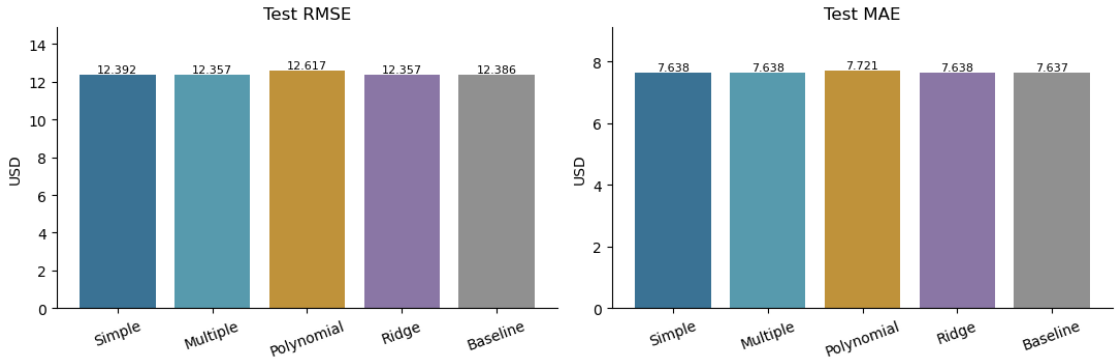
results = pd.DataFrame({name:evaluate(test.Next_Close,pred) for name,pred in␣
 ↪predictions.items()},
                       index=['Test R2','Test RMSE','Test MAE']).T
results['Train RMSE'] = pd.Series({name:evaluate(train.Next_Close,pred)[1] for␣
 ↪name,pred in fitted.items()})
results.loc['No-change baseline','Train RMSE'] = evaluate(train.
 ↪Next_Close,train.Close)[1]

print(results.round(5).to_string())

assert all(len(p)==len(test) and np.isfinite(p).all() for p in predictions.
 ↪values())
```

```
                    Test R2  Test RMSE  Test MAE  Train RMSE
Simple linear       0.97028   12.39179   7.63818     4.90928
Multiple linear     0.97045   12.35690   7.63837     4.89868
Polynomial degree 2 0.96919   12.61691   7.72103     4.89028
Ridge               0.97045   12.35690   7.63837     4.89868
No-change baseline  0.97031   12.38628   7.63660     4.91250
```

[15]:
```
fig, axes = plt.subplots(1, 2, figsize=(11, 3.7))
for ax, metric in zip(axes, ['Test RMSE', 'Test MAE']):
    ax.bar(range(5), results[metric], color=['#3a7294', '#579aad', '#bf923a',␣
 ↪'#8a76a5', '#909090'])
    ax.set_xticks(range(5), ['Simple', 'Multiple', 'Polynomial', 'Ridge',␣
 ↪'Baseline'], rotation=20)
    ax.set(ylabel='USD', title=metric, ylim=(0, results[metric].max() * 1.18))
    for i, value in enumerate(results[metric]): ax.text(i, value, f'{value:.
 ↪3f}', ha='center', va='bottom', fontsize=8)
fig.tight_layout()
plt.show()
```



The multiple linear model has the lowest test RMSE (USD 12.357), but the no-change baseline is very close behind (USD 12.386) and actually has the lowest MAE. The differences are just a few cents, so we can't say that any model really beats the baseline. Training errors are much smaller than test errors, mainly because prices in the test period are higher and more volatile, especially during the sharp increase of early 2020.

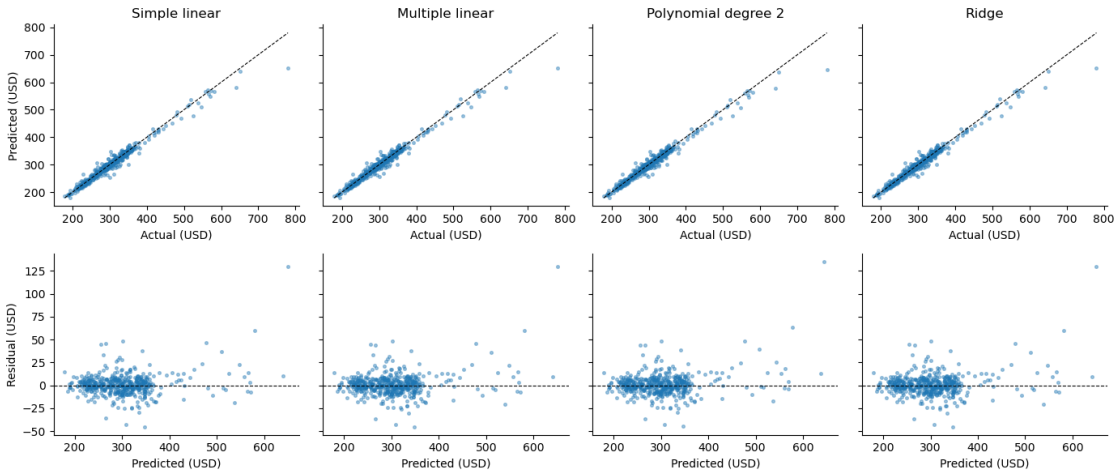
1.7 7. Model fit and residuals

In the top row we plot predicted against actual prices: if a model were perfect, all points would fall on the dashed diagonal. In the bottom row we plot the residuals (actual minus predicted) to look for patterns or changes in spread. All panels share the same axes so the models are easy to compare.

[16]:
```
names = list(specs) + ['Ridge']

fig, axes = plt.subplots(2, 4, figsize=(14, 6), sharey='row', sharex='row')
bounds = [min(test.Next_Close.min(), min(predictions[n].min() for n in names)),
          max(test.Next_Close.max(), max(predictions[n].max() for n in names))]

for i, name in enumerate(names):
    p = predictions[name]
    axes[0, i].scatter(test.Next_Close, p, s=7, alpha=0.4)
    axes[0, i].plot(bounds, bounds, 'k--', lw=0.8)
    axes[0, i].set(title=name, xlabel='Actual (USD)')
    axes[1, i].scatter(p, test.Next_Close.to_numpy()-p, s=7, alpha=0.4)
    axes[1, i].axhline(0, color='black', linestyle='--', lw=0.8)
    axes[1, i].set(xlabel='Predicted (USD)')
axes[0, 0].set_ylabel('Predicted (USD)')
axes[1, 0].set_ylabel('Residual (USD)')
fig.tight_layout()
plt.show()
```



All four models follow the general price level well, which is why the points stay close to the diagonal. However, at higher prices the residuals spread out a lot, and there are some large positive residuals, meaning the models underpredict when the price jumps suddenly. Adding squared terms (polynomial model) does not fix these errors. So, a high R^2 here does not mean the models can reliably forecast the market.

1.8 8. Supplementary analysis: the early-2020 price increase

Since the price shot up at the end of the test period, we wanted to know how much those days affect the overall error. We split the same test predictions into two groups: before 2020, and January 2020 to the last day (3 February 2020). We do not retrain any model or remove any data; we just look at the errors separately.

[17]:
```
# Diagnose the existing test predictions without refitting or deleting rows.
period_masks = {'Before 2020': (test.Target_Date < '2020-01-01').to_numpy(),
                'Jan-Feb 2020': (test.Target_Date >= '2020-01-01').to_numpy(),
                'Full test': np.ones(len(test), dtype=bool)}

actual = test.Next_Close.to_numpy()
rows = []

for name, pred in predictions.items():
    error = actual - pred
    rows.append({'Model': name, **{f'{p} RMSE': np.sqrt(np.mean(error[m]**2))␣
 ↪for p,m in period_masks.items()}})

period_table = pd.DataFrame(rows).set_index('Model')

print('Observations:', {p:int(m.sum()) for p,m in period_masks.items()}, '|␣
 ↪errors in USD')
```

```
display(period_table.round(3))

# Contribution of early 2020 to the multiple linear model's squared error
error = actual - predictions['Multiple linear']
m20 = period_masks['Jan-Feb 2020']

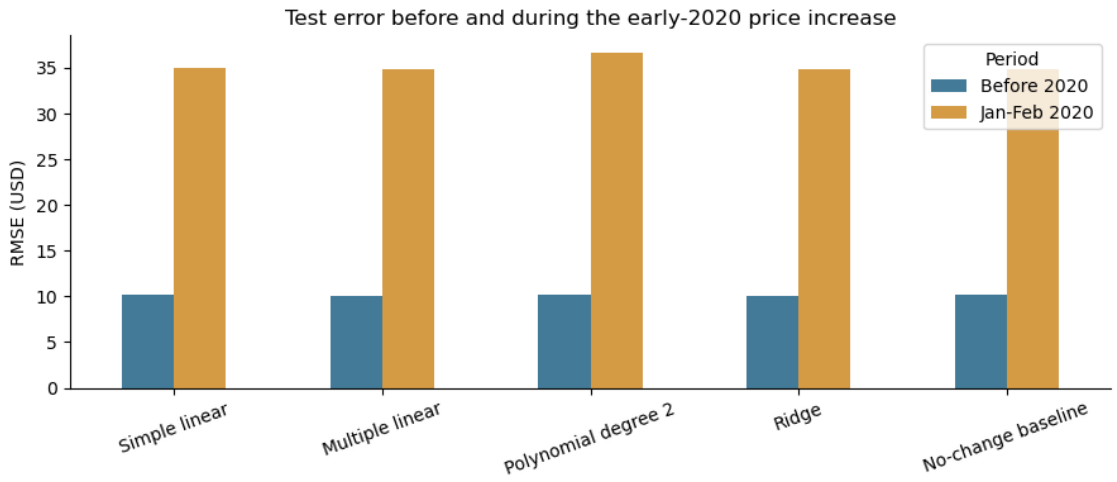
print(f'Early-2020 share of test observations: {100*m20.mean():.1f}%')
print(f'Early-2020 share of multiple-linear squared error: {100*np.
  ↪sum(error[m20]**2)/np.sum(error**2):.1f}%')
print(f'Maximum training close: USD {train.Close.max():.2f} | maximum test␣
  ↪close: USD {test.Next_Close.max():.2f}')
last = test.index[-1]
print(f'Last forecast ({test.Target_Date.iloc[-1].date()}): predicted USD␣
  ↪{predictions["Multiple linear"][-1]:.2f}, '
      f'actual USD {actual[-1]:.2f}, error USD {error[-1]:.2f}')
```

```
Observations: {'Before 2020': 461, 'Jan-Feb 2020': 22, 'Full test': 483} |
errors in USD
```

Model	Before 2020 RMSE	Jan-Feb 2020 RMSE	Full test RMSE
Simple linear	10.130	34.944	12.392
Multiple linear	10.102	34.844	12.357
Polynomial degree 2	10.122	36.712	12.617
Ridge	10.102	34.844	12.357
No-change baseline	10.133	34.880	12.386

```
Early-2020 share of test observations: 4.6%
Early-2020 share of multiple-linear squared error: 36.2%
Maximum training close: USD 385.00 | maximum test close: USD 780.00
Last forecast (2020-02-03): predicted USD 650.59, actual USD 780.00, error USD
129.41
```

[18]:
```
ax = period_table[['Before 2020 RMSE','Jan-Feb 2020 RMSE']].plot.
  ↪bar(figsize=(9,4),color=['#427a98','#d49a44'])
ax.set(xlabel='',ylabel='RMSE (USD)',title='Test error before and during the␣
  ↪early-2020 price increase')
ax.legend(['Before 2020','Jan-Feb 2020'], title='Period')
ax.tick_params(axis='x',rotation=20)
plt.tight_layout(); plt.show()
```



For the multiple linear model, RMSE goes from USD 10.10 before 2020 to USD 34.84 in the 22 days of early 2020. Those 22 days are only 4.6% of the test set, but they account for 36.2% of the total squared error. All models, including the baseline, show the same jump, so the problem is not specific to one method.

The highest closing price in the training data was USD 385, but in the test set prices reached USD 780. In other words, the models had to predict prices far outside the range they learned from. On 3 February 2020 the multiple model predicted USD 650.59, but the actual close was USD 780.00, a miss of USD 129.41. This analysis only shows where the errors happen; it does not explain why the price rose.

1.9 9. Extension: what can and cannot be predicted

The high R^2 of the previous models comes from predicting the price *level*, which is easy because tomorrow's price is always close to today's. A tougher and more useful test is whether the models can predict the daily *change*: how much and in which direction the price will move. Here we re-use the same test predictions and measure the error as a percentage of the price, how often the models guess the direction correctly, and the R^2 calculated on the daily change instead of the price.

[19]:
```
# How well do the models predict the daily CHANGE, not just the price level?
actual_next = test.Next_Close.to_numpy()
today = test.Close.to_numpy()
actual_change = actual_next - today
rows = []

for name, pred in predictions.items():
    if name == 'No-change baseline':
        continue  # the baseline always predicts zero change
    pred_change = pred - today
    rows.append({'Model': name,
```

```
                'MAE (% of price)': 100*np.mean(np.abs(actual_next - pred)/
 ↪actual_next),
                'Direction accuracy (%)': 100*np.mean((pred_change > 0) ==␣
 ↪(actual_change > 0)),
                'Predicted "up" days (%)': 100*np.mean(pred_change > 0),
                'R2 on daily change': 1 - np.sum((actual_change -␣
 ↪pred_change)**2)
                                          / np.sum((actual_change -␣
 ↪actual_change.mean())**2)})
change_table = pd.DataFrame(rows).set_index('Model')

print(f'Average absolute daily move in the test period: {100*np.mean(np.
 ↪abs(actual_change)/today):.2f}% of the price')
print(f'Days the price actually went up: {100*np.mean(actual_change > 0):.1f}%')

display(change_table.round(3))
```

```
Average absolute daily move in the test period: 2.48% of the price
Days the price actually went up: 50.1%
```

Model	MAE (% of price)	Direction accuracy (%)	Predicted "up" days (%)	R2 on daily change
Simple linear	2.458	48.033	92.961	-0.006
Multiple linear	2.461	49.275	63.561	-0.001
Polynomial degree 2	2.478	48.654	48.861	-0.043
Ridge	2.461	49.275	63.561	-0.001

The models miss the next-day price by about 2.5% on average, which is exactly the size of a typical daily move (2.48%). In other words, the error is as large as the movement we are trying to predict. The models guess the direction correctly only 48–49% of the time, no better than flipping a coin, even though the price went up on 50.1% of the days. The simple linear model, for example, predicts "up" on 93% of the days simply because its slope is close to 1 and its intercept is positive. The R^2 on the daily change is about zero (slightly negative) for every model, so they explain none of the day-to-day movement. This is consistent with the idea that stock prices behave almost like a random walk: with this information, the best forecast of tomorrow's price is today's price. If the direction of the move cannot be predicted, maybe its *size* can. Stock market volatility tends to cluster: calm days are usually followed by calm days, and turbulent days by turbulent days. To test this, we predict tomorrow's daily range (high minus low, as a percentage of the price) using today's range, the average range over the last 5 and 20 days, the average size of recent price moves and the trading volume (in log scale). We use the same chronological 80/20 split and compare against two simple baselines: tomorrow's range equals today's, and tomorrow's range equals the 20-day average.

[20]:
```
# Can we predict how much the price will move tomorrow (volatility)?
vol = df[['Date','Close','High','Low','Volume']].copy()
vol['Range_pct'] = 100*(vol.High - vol.Low)/vol.Close        # daily range as %␣
 ↪of the price
vol['Log_ret'] = 100*np.log(vol.Close).diff()                # daily return in %
vol['Range_5d'] = vol.Range_pct.rolling(5).mean()            # average range of␣
 ↪the last 5 days
vol['Range_20d'] = vol.Range_pct.rolling(20).mean()          # average range of␣
 ↪the last 20 days
vol['AbsRet_5d'] = vol.Log_ret.abs().rolling(5).mean()       # average size of␣
 ↪recent moves
vol['Log_volume'] = np.log(vol.Volume)                       # logarithm of the␣
 ↪volume
vol['Next_Range_pct'] = vol.Range_pct.shift(-1)              # target:␣
 ↪tomorrow's range
vol = vol.dropna().reset_index(drop=True)
vol_features = ['Range_pct','Range_5d','Range_20d','AbsRet_5d','Log_volume']

# Same chronological 80/20 split with a one-row gap
vcut = int(len(vol)*0.8)
vtrain, vtest = vol.iloc[:vcut-1], vol.iloc[vcut:]
vol_model = sm.OLS(vtrain.Next_Range_pct, sm.
 ↪add_constant(vtrain[vol_features])).fit()
vol_pred = vol_model.predict(sm.add_constant(vtest[vol_features])).to_numpy()

vol_results = pd.DataFrame({name: evaluate(vtest.Next_Range_pct, p) for name, p␣
 ↪in {
        'OLS volatility model': vol_pred,
        'Baseline: same range as today': vtest.Range_pct.to_numpy(),
        'Baseline: 20-day average range': vtest.Range_20d.to_numpy()}.items()},
    index=['Test R2','Test RMSE (%)','Test MAE (%)']).T

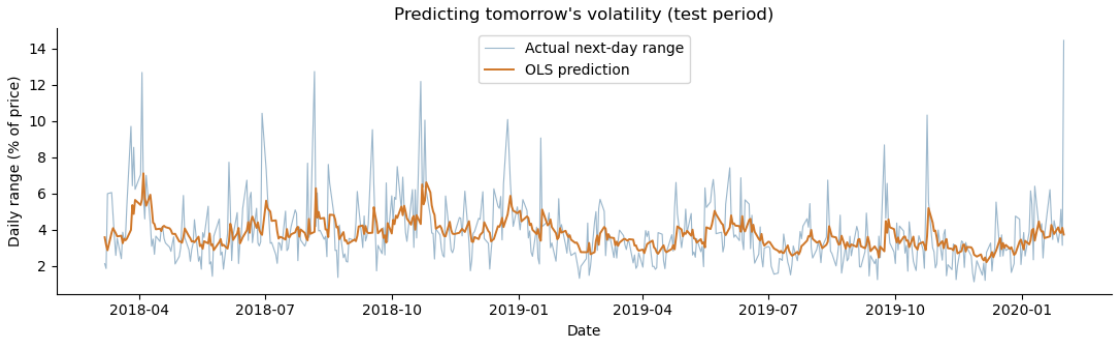
print('Training observations:', len(vtrain), '| Test observations:', len(vtest))
display(vol_results.round(3))
```

```
Training observations: 1915 | Test observations: 480
```

	Test R2	Test RMSE (%)	Test MAE (%)
OLS volatility model	0.156	1.660	1.173
Baseline: same range as today	-0.177	1.960	1.385
Baseline: 20-day average range	0.022	1.787	1.310

[21]:
```
fig, ax = plt.subplots(figsize=(11,3.5))
ax.plot(vtest.Date, vtest.Next_Range_pct, lw=.8, color='#9bb7cc', label='Actual␣
 ↪next-day range')
ax.plot(vtest.Date, vol_pred, lw=1.4, color='#d0782a', label='OLS prediction')
```

```
ax.set(xlabel='Date', ylabel='Daily range (% of price)', title='Predicting␣
 ↪tomorrow\'s volatility (test period)')
ax.legend(); fig.tight_layout(); plt.show()
```



Here the picture is different: the volatility model reaches a test R^2 of 0.156 and beats both baselines (RMSE 1.66% vs. 1.79% for the 20-day average and 1.96% for "same as today"). The plot shows that the prediction follows the calmer and more turbulent periods, although it cannot anticipate the sudden spikes. An R^2 of 0.16 may look low, but for daily financial data it is a meaningful result, because it shows there is real, usable information in recent volatility. So, with this data we cannot predict where Tesla's price is going tomorrow, but we can partly predict how much it is likely to move, which is useful for managing risk.

1.10 10. Conclusions

- **What we have.** Tesla stock dataset from 2012 to 2020. The data needed no cleaning, and after creating the target we had 2,415 days (1,931 for training and 483 for testing).
- **What we predicted.** Tesla's closing price for the next trading day, using today's close, volume and daily range.
- **What the exploration showed.** All the price columns move together almost perfectly, so we used only `Close`. Volume and range are only moderately related to the target.
- **How the models did.** All models reached a test R^2 of about 0.97. Multiple linear regression had the lowest RMSE (USD 12.357), and Ridge gave the same result because the best penalty was almost zero; with only three predictors there was little to regularize. The polynomial model fit the training data slightly better but did worse on the test set, a small sign of overfitting.
- **The main lesson.** The no-change baseline was practically as good as any model (RMSE USD 12.386 and the best MAE). The best model only improves RMSE by about 3 cents. The high R^2 comes from the fact that today's price is very close to tomorrow's, not from an ability to predict daily movements.
- **Early 2020.** The last 22 test days produced over a third of the squared error, because prices went well above anything seen in training.
- **What can be predicted.** When we measure the daily change instead of the price level, the models explain nothing (R^2 0) and guess the direction only about 49% of the time. However, tomorrow's volatility (daily range) is partly predictable from recent volatility (test $R^2 = 0.16$), beating simple baselines.

- **What we would try next.** Predict daily returns (percentage changes) instead of prices, add lagged values or market indicators, retrain the model as new data arrives (walk-forward validation), and test it on more recent Tesla data. Furthermore, more advanced machine learning models can be used, as neural networks or random forrest.

1.11 Sources

- Tesla Stock Data from 2010 to 2020, Kaggle, user *timoboz*: https://www.kaggle.com/timoboz/tesla-stock-data-from-2010-to-2020
- DATA 555 course materials: Module 5 (Data Cleaning) and Module 6 (Basic Predictions) notebooks.
- Library documentation: pandas, seaborn, statsmodels and scikit-learn.